7. JSX

▼ JSX

- JSX stands for **JavaScript** and **XML**.
- It is a syntax extension for JavaScript that looks like HTML but works inside JavaScript.
- This syntax allow us to write html code in react.
- It looks similar to html but it is not.
- Without jsx it is very difficult to create react application.
- **Babel JS** is internally converts JSX to browser-understandable JavaScript.

▼ Without Using JSX (Native Elements)

- **For Example →**

```
import React from "react";
import ReactDOM from "react-dom/client";

let element = React.createElement("h1", null, "I am an element without JSX");
console.log(element); // returns an object

ReactDOM.createRoot(document.getElementById("root")).render(element);
```

- **Return Type**: It will return an object with multiple properties.
- `$$typeof`: It is the property to identify that this is a react element. This is the special property for safety and security reasons. This works because you can't just put symbols in JSON.

▼ With JSX

- **For Example →**

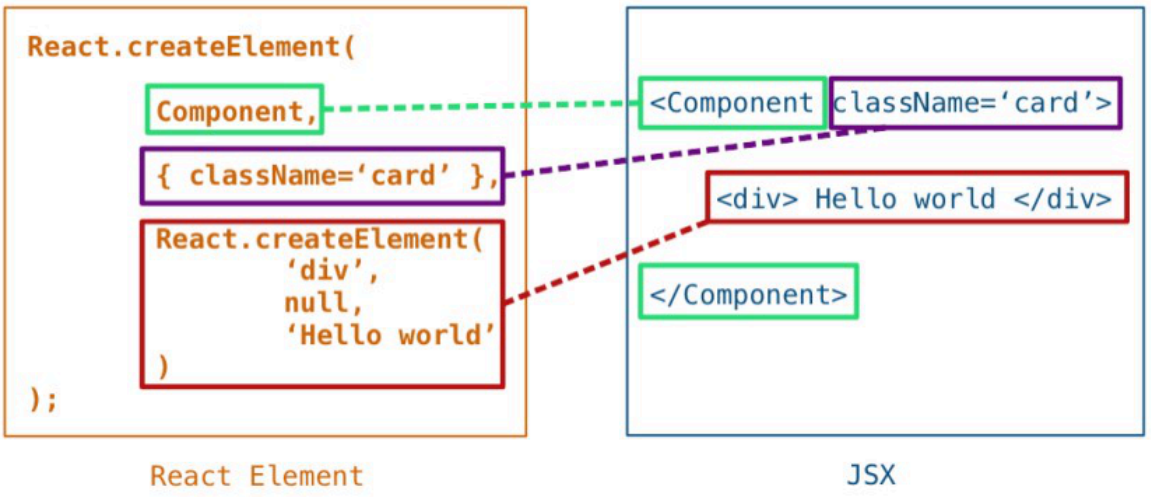
```
import React from "react";
import ReactDOM from "react-dom/client";

let elementWithJsx = <h1>I am JSX</h1>;
console.log(elementWithJsx); // returns an object

ReactDOM.createRoot(document.getElementById("root")).render(elementWithJsx);
```

- The browser understands both of them as React element objects, whether it is a native element or JSX.

> NOTE: Internally, react uses native elements. For development, we should be using JSX.

▼ React.createElement() vs JSX

- The following image shows how to compose React Element from ***React.createElement*** method and ***JSX.***



▼ JSX Rules

▼ Rule-1: JSX must have one parent element

- React components must return a **single root element**.

- Because JSX converted to `React.createElement()`, which must return **one parent node**.
- If you want to return multiple elements, wrap them in a single parent tag/element.
- Same level (siblings) elements, if you are having then you should wrap them in a parent element. (like `<div>`, `<section>`, or a `ReactFrgament`)
- **For Example** →
- **Incorrect**: It will throw an error called - JSX expressions must have one parent element.

```
import React from 'react'

const App = () => {
  return (
    <h1>First Element</h1> //error: JSX expressions must have one pa
rent element.
    <h2>Second Element</h2>
  )
}

export default App
```

- **Correct Way** → Wrap both the child elements by one parent element.

```
import React from "react";

const App = () => {
  return (
    //Here, div is the parent element for wrapper
    <div>
      <h1>First Element</h1>
      <h2>Second Element</h2>
    </div>
  );
};
```

```
export default App;
```

▼ Rule-2 : Close All Tags in JSX

- JSX, the syntax extension used in React, is stricter than HTML.
- It follows strict rules like XML, requiring that **all the tags be explicitly closed**.

  1. All tags must be closed.
  2. Use a forward slash (`/`) before the closing of angular bracket for self-closing tags.
  3. Common self-closing tags include: `<img/>`, `<input/>`, `<br/>`, `<hr/>`, `<meta/>`, `<link/ >`, etc.
  4. **For Example** →
  5. **Incorrect Way**:

```
import React from "react";

const App = () => {
  return (
    <div>
     <img src="logo.png" alt="Logo">//error: JSX element 'img' h
as no corresponding closing tag.
      <input type="text">
      <br>
      <hr>
    </div>
  );
};

export default App;
```

  6. **Correct Way:**

```
import React from "react";
```

```
const App = () => {
  return (
    <div>
      <img src="logo.png" alt="Logo" />
      <input type="text" />
      <br />
      <hr />
    </div>
  );
};

export default App;
```

▼ Rule-3 : Wrap Multi-line JSX in Parentheses ()

- In React, when writing JSX that takes mulitple lines, its best practice to wrap the JSX code in parentheses ().
- **For Example** →
- **Incorrect**:

```
import React from "react";

const App = () => {
  return
    //Unreachable code detected.
    <div>
      <h1>Hello, World!</h1>
      <p>Welcome to React.</p>
    </div>
};

export default App;
```

- **Correct**:

```
import React from "react";
```

```
const App = () => {
  return (
    // Use parentheses
    <div>
      <h1>Hello, World!</h1>
      <p>Welcome to React.</p>
    </div>
  );
};

export default App;
```

▼ `Rule-4` : Use `{}` for JavaScript Expressions in JSX

▼ What is a JSX Expression in React?

- If you want to write the JavaScript code into your JSX (HTML structure), we have to use **curly braces** `{}`.
- This embedded JavaScript is called a **JSX Expression**.

> 💡 An expression is a piece of code that evaluates to a value - like a variable, arithmetic operation, function call, ternary condition, etc.

- **For Example** →

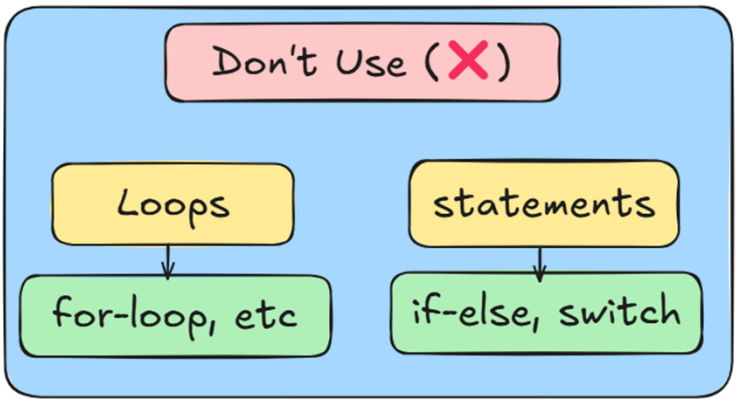
```
import React from "react";

const App = () => {
  let name = "React";
  return (
    <div>
      <h1>Name: {name}</h1>
    </div>
  );
};
```

```
export default App;
```

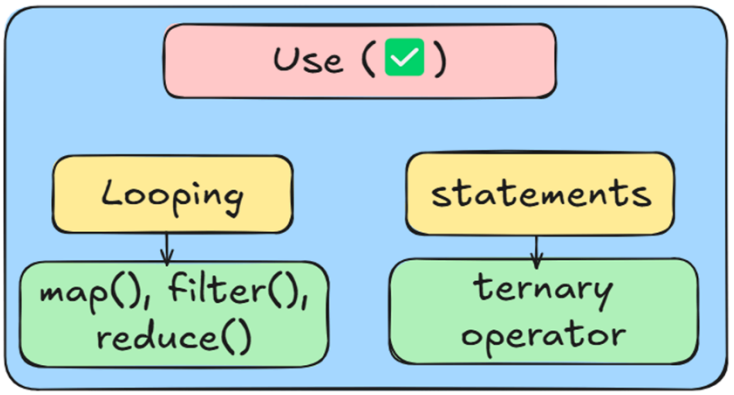
> `Important Note`: Inside expression we cannot render `boolean/null/undefined` but we can use with conditions.

▼ What is not allowed in JSX Expression?

- You cannot use statements inside the JSX Expression.



- You can use the expressions inside the JSX Expression.



▼ `Rule-5`: JSX attributes Use Camelcase Convention

- In JSX, HTML attributes must be written in **camelcase** instead of lowercase.
- This is because JSX is closer to **JavaScript Objects** than HTML syntax.

- **For Example →**

```
// HTML
<input type="text" onclick="doSomething()" />

// JSX
<input type="text" onClick={doSomething} />
```

▼ `Rule-6` : Use `className` Instead of `class`

- In JSX, `class` is a **reserved keyword in JavaScript** (used for defining class).
- So, React uses `className` instead of `class` to apply CSS classes.
- **For Example →**

```
// HTML
<div class="container"></div>

// JSX
<div className="container"></div>
```

▼ `Rule-7` : Use `htmlFor` Instead of `for` in Label tag

- In JSX, `for` is also a **JavaScript keyword** (used in loops).
- So, React replaces it with `htmlFor` to label form elements correctly.
- **For Example →**

```
// HTML
<label for="email">Email</label>
<input id="email" />

// JSX
<label htmlFor="email">Email</label>
<input id="email" />
```

▼ `Rule-8` : Custom Components Should Be in PascalCase

- In JSX, **user-defined React components** must start with an **uppercase letter** (usually **PascalCase**).
- Lowercase tag names are treated as **native HTML elements** like `<div>`, `<input>`.
- **Correct (PascalCase):**

```
function MyComponent() {
  return <h1>Hello from MyComponent</h1>;
}

<MyComponent />
```

- **Incorrect (lowercase):**

```
function mycomponent() {
  return <h1>Hello</h1>;
}

<mycomponent />  // ❌ React will treat this as a custom HTML tag and throw a warning.
```